



Zigzag Traversal (medium)

We'll cover the following ^

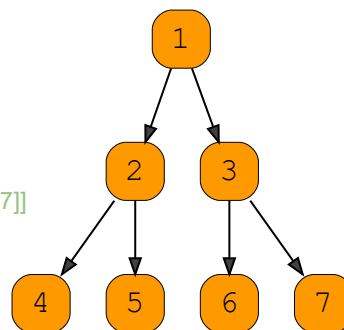
- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity

Problem Statement

Given a binary tree, populate an array to represent its zigzag level order traversal. You should populate the values of all **nodes of the first level from left to right**, then **right to left for the next level** and keep alternating in the same manner for the following levels.

Example 1:

Zigzag Level Order Traversal: `[[1],
[3, 2],
[4, 5, 6, 7]]`

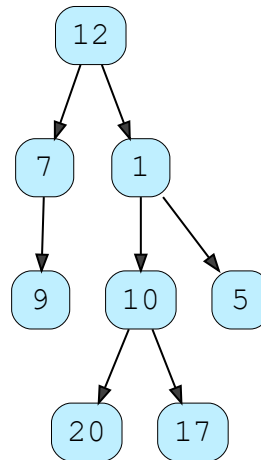


Example 2:



Zigzag Level Order Traversal:

[[12],
[1,7],
[9,10,5],
[17,20]]



Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class TreeNode {
4     int val;
5     TreeNode left;
6     TreeNode right;
7
8     TreeNode(int x) {
9         val = x;
10    }
11 };
12
13 class ZigzagTraversal {
14     public static List<List<Integer>> traverse(TreeNode root) {
15         List<List<Integer>> result = new ArrayList<List<Integer>>();
16         // TODO: Write your code here
17         return result;
18     }
19
20     public static void main(String[] args) {
21         TreeNode root = new TreeNode(12);
22         root.left = new TreeNode(7);
23         root.right = new TreeNode(1);
24         root.left.left = new TreeNode(9);
25         root.right.left = new TreeNode(10);
26         root.right.right = new TreeNode(5);
27         root.right.left.left = new TreeNode(20);
28         root.right.left.right = new TreeNode(17);
29         List<List<Integer>> result = ZigzagTraversal.traverse(root);
30         System.out.println("Zigzag traversal: " + result);
31     }
32 }
33
```

Run

Save

Reset



Solution

This problem follows the [Binary Tree Level Order Traversal](#) pattern. We can follow the same **BFS** approach. The only additional step we have to keep in mind is to alternate the level order traversal, which means that for every other level, we will traverse similar to [Reverse Level Order Traversal](#).

Code

Here is what our algorithm will look like, only the highlighted lines have changed:

Java

Python3

C++

JS

```
1 import java.util.*;
2
3 class TreeNode {
4     int val;
5     TreeNode left;
6     TreeNode right;
7
8     TreeNode(int x) {
9         val = x;
10    }
11 };
12
13 class ZigzagTraversal {
14     public static List<List<Integer>> traverse(TreeNode root) {
15         List<List<Integer>> result = new ArrayList<List<Integer>>();
16         if (root == null)
17             return result;
18
19         Queue<TreeNode> queue = new LinkedList<>();
20         queue.offer(root);
21         boolean leftToRight = true;
22         while (!queue.isEmpty()) {
23             int levelSize = queue.size();
24             List<Integer> currentLevel = new LinkedList<>();
25             for (int i = 0; i < levelSize; i++) {
26                 TreeNode currentNode = queue.poll();
27
28                 // add the node to the current level based on the traverse direction
29                 if (leftToRight)
30                     currentLevel.add(currentNode.val);
31                 else
32                     currentLevel.add(0, currentNode.val);
33
34                 // insert the children of current node in the queue
35                 if (currentNode.left != null)
36                     queue.offer(currentNode.left);
37                 if (currentNode.right != null)
38                     queue.offer(currentNode.right);
39             }
40             result.add(currentLevel);
41             // reverse the traversal direction
42             leftToRight = !leftToRight;
43         }
44
45         return result;
46     }
47
48     public static void main(String[] args) {
49         TreeNode root = new TreeNode(12);
50         root.left = new TreeNode(7);
51         root.right = new TreeNode(1);
52         root.left.left = new TreeNode(9);
```

```
53     root.right.left = new TreeNode(10);
54     root.right.right = new TreeNode(5);
55     root.right.left.left = new TreeNode(20);
56     root.right.left.right = new TreeNode(17);
57     List<List<Integer>> result = ZigzagTraversal.traverse(root);
58     System.out.println("Zigzag traversal: " + result);
59 }
60 }
61
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ as we need to return a list containing the level order traversal. We will also need $O(N)$ space for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.